

# Paper overview

## CVAE-GAN: Fine-Grained Image Generation through Asymmetric Training

Jianmin Bao, Dong Chen, Fang Wen, Houqiang Li, Gang Hua

ICCV 2017, Venice, Italy DOI: 10.1109/ICCV.2017.299

- ▶ **Problem:** conditional generators struggle to produce images that are realistic, diverse, and faithful to a fine-grained class.
- ▶ **Motivation:** VAE outputs tend to blur; GAN training can suffer from unstable gradients and mode collapse.
- ▶ **Main contribution:** a conditional VAE-GAN with asymmetric objectives, mean feature matching, and pairwise reconstruction constraints.
- ▶ **Project code:** <https://github.com/sivajeetsabdakar/cvae-gan-pytorch-reproduction>

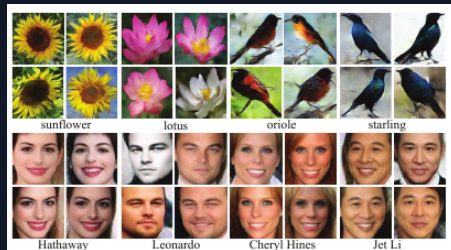
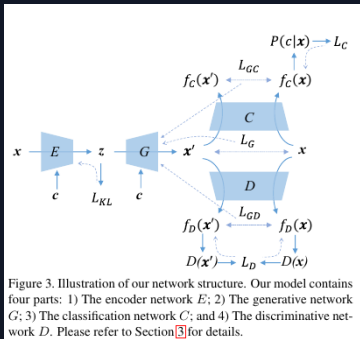


Figure 1. **Synthesized images** using our CVAE-GAN model at high resolution ( $128 \times 128$ ) for different classes. The generated samples are realistic and diverse within a class.

Paper Figure 1:  $128 \times 128$  class-conditioned samples.

# Methodology



Paper Figure 3: encoder  $E$ , generator  $G$ , classifier  $C$ , discriminator  $D$ .

- **Workflow:**  $E(x, c)$  infers latent  $z$ ;  $G(z, c)$  reconstructs or samples  $x'$ ;  $C$  preserves class identity;  $D$  enforces realism.
- **Datasets:** FaceScrub, Oxford 102 Flowers, and CUB-200 birds.
- **Generator objective:** combine pixel reconstruction, discriminator-feature matching, and classifier-feature matching.
- **Asymmetric training:**  $C$  and  $D$  use cross-entropy, while  $G$  matches feature means with an  $\ell_2$  objective.
- **Key settings:**  $128 \times 128$  images, 256-D latent code, batch normalization,  $\lambda_1 = 3$ ,  $\lambda_2 = 1$ ,  $\lambda_3 = \lambda_4 = 10^{-3}$ .

# Reproduction and implementation

**Algorithm 1** The training pipeline of the proposed CVAE-GAN algorithm.

**Require:**  $m$ , the batch size.  $n$ , class number.  $\theta_E$ , initial  $E$  network parameters.  $\theta_G$ , initial  $G$  network parameters.  $\theta_D$ , initial  $D$  network parameters.  $\theta_C$ , initial  $C$  network parameters,  $\lambda_1 = 3$ ,  $\lambda_2 = 1$ ,  $\lambda_3 = 10^{-3}$  and  $\lambda_4 = 10^{-3}$ .

```
1: while  $\theta_G$  has not converged do
2:   Sample  $\{x_r, c_r\} \sim P_r$  a batch from the real data;
3:    $\mathcal{L}_C \leftarrow -\log(P(c_r|x_r))$ 
4:    $z \leftarrow E(x_r, c_r)$ 
5:    $\mathcal{L}_{KL} \leftarrow KL(q(z|x_r, c_r)||P_z)$ 
6:    $x_f \leftarrow G(z, c_r)$ 
7:   Sample  $z_p \sim P_z$  a batch of random noise, sample  $c_p$  a
   batch of random classes;
8:    $x_p \leftarrow G(z_p, c_p)$ 
9:    $\mathcal{L}_D \leftarrow -(\log(D(x_r)) + \log(1 - D(x_f)) + \log(1 - D(x_p)))$ 
10:  Calculate  $x_r$  feature center  $\frac{1}{m} \sum_i^m f_D(x_r)$  and  $x_p$  feature
   center  $\frac{1}{m} \sum_i^m f_D(x_p)$ ;
11:   $\mathcal{L}_{GD} \leftarrow \frac{1}{2} \|\frac{1}{m} \sum_i^m f_D(x_r) - \frac{1}{m} \sum_i^m f_D(x_p)\|_2^2$ 
12:  Calculate each class  $c_i$  feature center  $f_C^{c_i}(x_r)$  for  $x_r$  and
    $f_C^{c_i}(x_p)$  for  $x_p$  using moving average method;
13:   $\mathcal{L}_{GC} \leftarrow \frac{1}{2} \sum_{c_i} \|f_C^{c_i}(x_r) - f_C^{c_i}(x_p)\|_2^2$ 
14:   $\mathcal{L}_G \leftarrow \frac{1}{2} (\|x_r - x_f\|_2^2 + \|f_D(x_r) - f_D(x_f)\|_2^2 + \|f_C(x_r) -
   f_C(x_f)\|_2^2)$ 
15:   $\theta_C \leftarrow \theta_C^+ - \nabla_{\theta_C}(\mathcal{L}_C)$ 
16:   $\theta_D \leftarrow \theta_D^+ - \nabla_{\theta_D}(\mathcal{L}_D)$ 
17:   $\theta_G \leftarrow \theta_G^+ - \nabla_{\theta_G}(\lambda_2 \mathcal{L}_G + \lambda_3 \mathcal{L}_{GD} + \lambda_4 \mathcal{L}_{GC})$ 
18:   $\theta_E \leftarrow \theta_E^+ - \nabla_{\theta_E}(\lambda_1 \mathcal{L}_{KL} + \lambda_2 \mathcal{L}_G)$ 
19: end while
```

Paper Algorithm 1: alternating updates for  $C$ ,  $D$ ,  $G$ , and  $E$ .

- **Original toolchain:** Torch; GoogLeNet encoder, six-layer deconvolutional generator, DCGAN discriminator, AlexNet classifier.
- **Dataset preparation:** align FaceScrub faces from five landmarks; crop Oxford Flowers using masks; retain original CUB-200 images; resize inputs to  $128 \times 128$ .
- **Implementation approach:** alternate network updates exactly as Algorithm 1 and use moving averages for class-specific feature centers.
- **Experimental setup:** train all networks from scratch; sample  $z \sim \mathcal{N}(0, I)$  for category-conditioned testing.
- **Implementation status:** PyTorch code, shape/loss tests, checkpointing, sampling, and a one-step CPU smoke run are verified. Paper-scale training remains **Pending**.

# Results and comparison



Paper Figure 5: real samples, CVAE, CGAN, FM-CGAN, and CVAE-GAN. CVAE-GAN retains category identity and within-class diversity more consistently than the baselines shown.

## Original quantitative results

| Method    | Top-1 acc.    | Realism      |
|-----------|---------------|--------------|
| Real data | 99.61%        | 20.85        |
| CVAE      | 8.09%         | 10.29        |
| CGAN      | 61.97%        | 15.79        |
| FM-CGAN   | 79.76%        | <b>19.40</b> |
| CVAE-GAN  | <b>97.78%</b> | 19.03        |

## Original vs. reproduced

**Original evaluation:** 53k generated faces.

**Smoke reproduction:** training and sampling passed.

**Paper-scale metrics:** **Pending**; not comparable yet.

Metrics reproduced from paper Table 1. "Realism" is the paper's Inception Score variant.

# Conclusion and learning

- ▶ **Reproduction status:** implementation and end-to-end smoke validation succeeded; dataset-scale training and metric replication remain **Pending**.
- ▶ **Major finding:** feature matching stabilizes conditional GAN training, while the encoder and pairwise losses protect diversity and class structure.
- ▶ **Reported impact:** synthetic identity augmentation improved LFW verification from 91.87% to 92.98% in the paper.
- ▶ **Challenges:** legacy Torch code, three datasets with different preprocessing, four-network optimization, and evaluation with a separate classifier.
- ▶ **Next improvements:** implement in PyTorch, record seeds and checkpoints, compare FID/KID alongside the legacy score, and run ablations for each loss term.
- ▶ **Takeaway:** CVAE-GAN ties each latent code to a real example and trains the generator with feature statistics instead of the discriminator's cross-entropy alone.

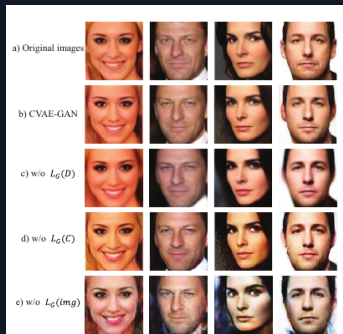


Figure 8. Visualization comparison between different generator  $G$ , each trained with different combination of losses.

| Method                           | Training Data | Accuracy |
|----------------------------------|---------------|----------|
| no data augmentation             | 80K           | 91.87%   |
| existing identities augmentation | 80K + 100K    | 92.77%   |
| 5k new identities augmentation   | 80K + 500K    | 92.98%   |

Table 2. Results of face data augmentation.

Paper Figures 7–8 and Table 2: inpainting, loss ablations, and face augmentation.